

Companion to Bird & de Moor, Algebra of Programming

§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- (9.1.1a)
- $\langle T \rangle = \text{concat}$
 $T=[\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
 - $\langle h \rangle = \text{list}(\text{sum})$
 $h=[\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
 - $H = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

	x	h(x)
1	$l(())$	$[]$
2	$r([c], [])$	$[c]$
3	$r([a, b], [c])$	$[a + b, c]$
4	$r([a], [b + c])$	$[a, b + c]$

- (9.1.1c)
- $H([a, b, c], ys) \iff ys = [a + b + c] \vee ys = [a, b + c] \vee ys = [a + b, c] \vee ys = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
 - $T^\circ F(H) h = H$
cut once, solve the rest, put the pieces back together

(9.1.1d)

	name	type	definition	meaning
1	H	A→B	$H \triangleq \langle T \rangle^\circ \langle h \rangle$	take the input apart by the T-decompositions, and put it together by h
2	M	A→B	$M \triangleq \frac{H}{\exists} \text{est}(R)$	the problem to be solved: an answer of H that is best under R

$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies (\mu X : \frac{T^\circ}{\exists} P(F(X) h) \text{est}(R)) \sqsubseteq \frac{H}{\exists} \text{est}(R)$

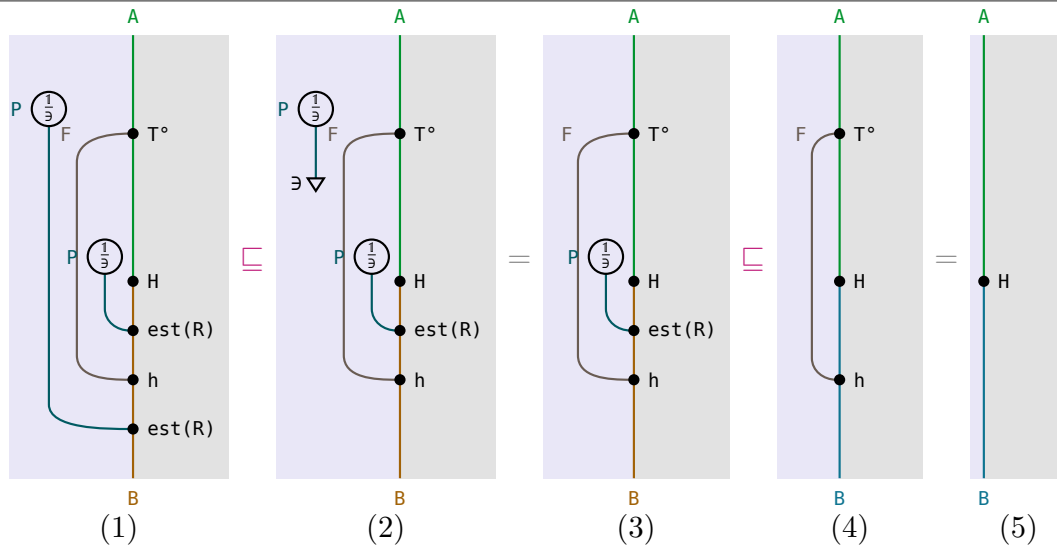
Theorem 9.1

every answer the recursion returns is an optimal one

- $\frac{T^\circ}{\exists}$ takes the input apart one step every way; $F(X)$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(R)$ keeps a best one
- $\sqsubseteq M$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H \quad (9.2)$$

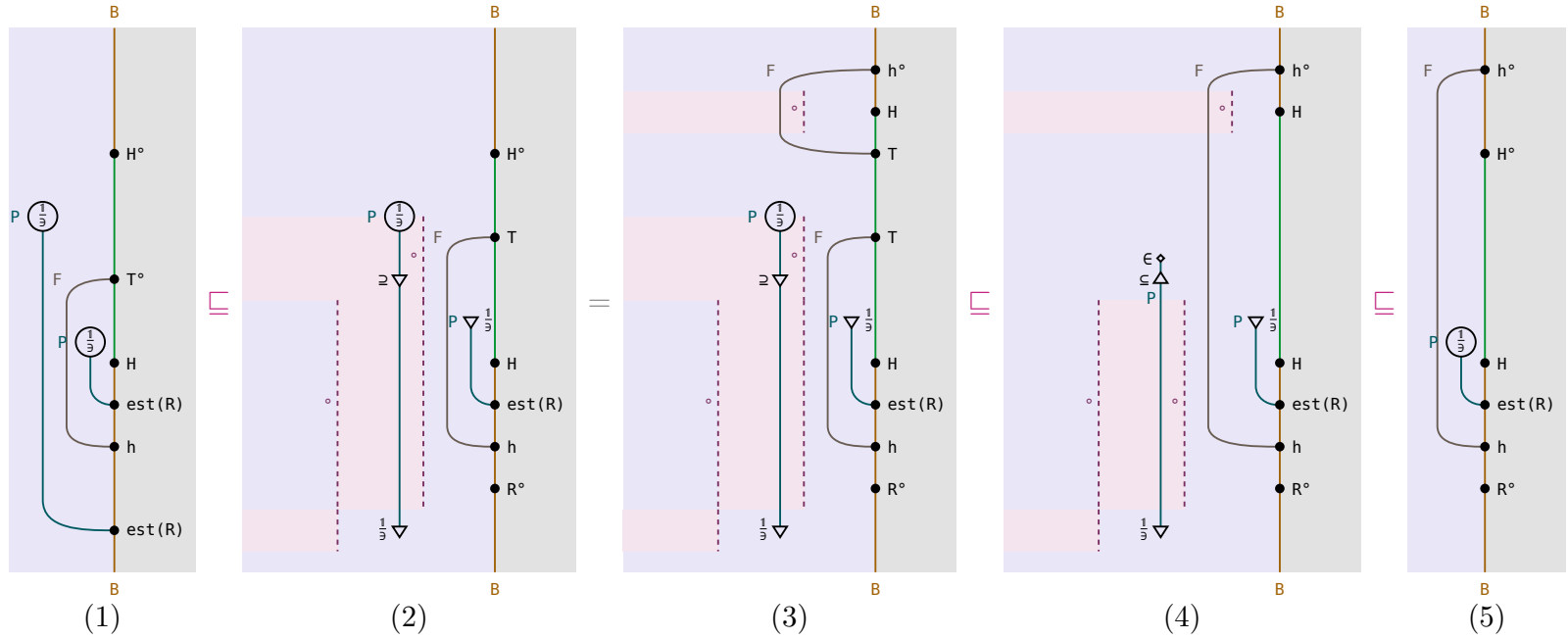
taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$)
returns only what H returns



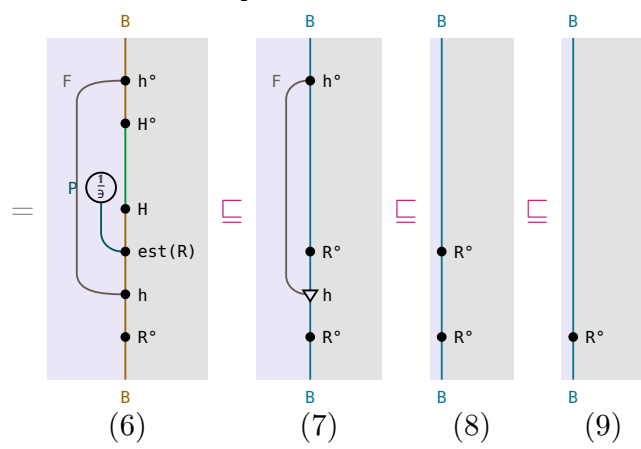
1	$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
2	$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{R}{\exists} \exists = R$
3	$= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{S}{\exists} \text{est}(R) \sqsubseteq S$
4	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
5	$= H$	

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every $\mathbf{b}$ that $\mathbf{H}$ returns from an input, the step $\frac{T^\circ}{\exists} P(F(\mathbf{M})h) \text{est}(R)$ returns from that input only $\mathbf{b}'$ with R relating $\mathbf{b}'$ to $\mathbf{b}$



1	$H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \epsilon \setminus (S R^\circ)$
2	$\sqsubseteq H^\circ \frac{T^\circ}{\exists} (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$H^\circ = h^\circ F(H^\circ) T$
3	$= h^\circ F(H^\circ) T \frac{T^\circ}{\exists} (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$T \frac{T^\circ}{\exists} \sqsubseteq \epsilon$
4	$\sqsubseteq h^\circ F(H^\circ) \epsilon (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$S (S \setminus R) \sqsubseteq R$
5	$\sqsubseteq h^\circ F(H^\circ) F(\frac{H}{\exists} \text{est}(R)) h R^\circ$	

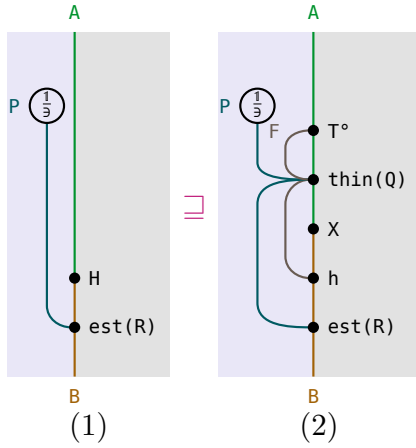


6	$= h^\circ F(H^\circ \frac{H}{\exists} \text{est}(R)) h R^\circ$	$S^\circ \frac{S}{\exists} \text{est}(R) \sqsubseteq R^\circ$
7	$\sqsubseteq h^\circ F(R^\circ) h R^\circ$	$\text{Map}(f)$
8	$\sqsubseteq R^\circ R^\circ$	$\implies f \text{ Pres } R \iff f^\circ F(R) f \sqsubseteq R$
9	$\sqsubseteq R^\circ$	$R^\circ R^\circ \sqsubseteq R^\circ$

Map(h) $\wedge$ h Pres $R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^\circ}{\exists} \text{thin}(Q) P(F(X) h) \text{est}(R)) \sqsubseteq \frac{H}{\exists} \text{est}(R)$ Theorem 9.2

an optimum over everything **H** returns is reached by taking the input apart every way **T** allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

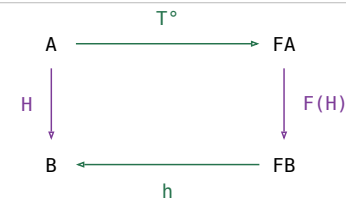
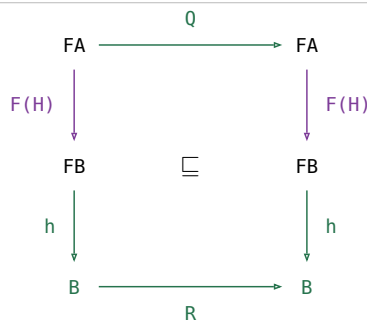
$$H \triangleq (T)^\circ (h) \text{ — } H \triangleq (T)^\circ (h)$$



$$\frac{H}{\exists} \text{est}(R)$$

$$\sqsubseteq \frac{T^\circ}{\exists} \text{thin}(Q) P(F(X) h) \text{est}(R)$$

$QF(H)h \sqsubseteq F(H)hR$, h monotonic on $R \text{ — } Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge R$
 $R \sqsubseteq R$
 $\Rightarrow \text{est}(R) = \text{thin}(Q) \text{est}(R), X \sqsubseteq \text{est}(R) \iff X \sqsubseteq \exists$
 $\wedge \in X \sqsubseteq R^\circ$



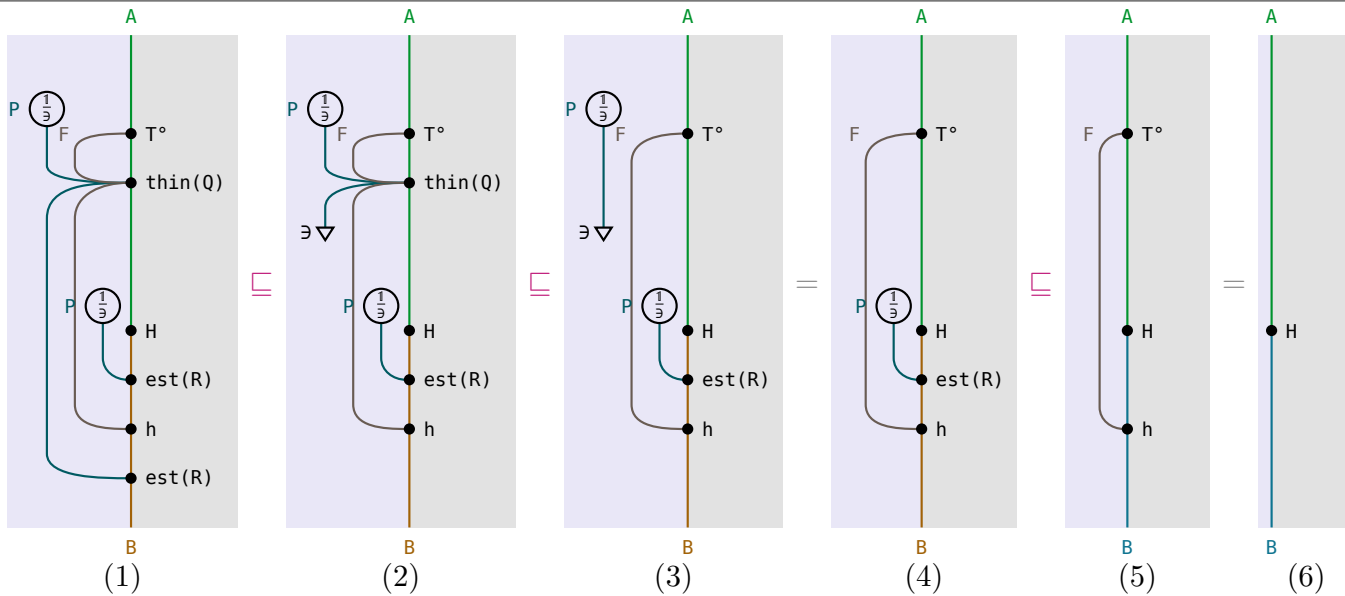
$T^\circ F(H)h = H$: taking the input apart once by T° , solving the parts by $F(H)$ and assembling by h returns exactly what H returns

$QF(H)h \sqsubseteq F(H)hR$: every result of thinning by Q , solving the parts by $F(H)$ and assembling by h is reached by R from a result of solving by $F(H)$ and assembling by h without Q

$$\frac{T^\circ}{\exists} \text{thin}(Q) P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H$$

(9.1.1e)

taking the input apart every way **T** allows ($\frac{T^\circ}{\exists}$), dropping the parts **thin(Q)** rejects, solving each rest by **M** and keeping an optimum returns only what **H** returns



$$\frac{T^\circ}{\exists} \text{thin}(Q) P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$$

$$\sqsubseteq \frac{T^\circ}{\exists} \text{thin}(Q) \exists F(\frac{H}{\exists} \text{est}(R)) h$$

$$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$$

$$= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$$

$$\sqsubseteq T^\circ F(H) h$$

$$= H$$

$$P(S) \text{est}(R) \sqsubseteq \exists S$$

$$\text{thin}(Q) \exists \sqsubseteq \exists$$

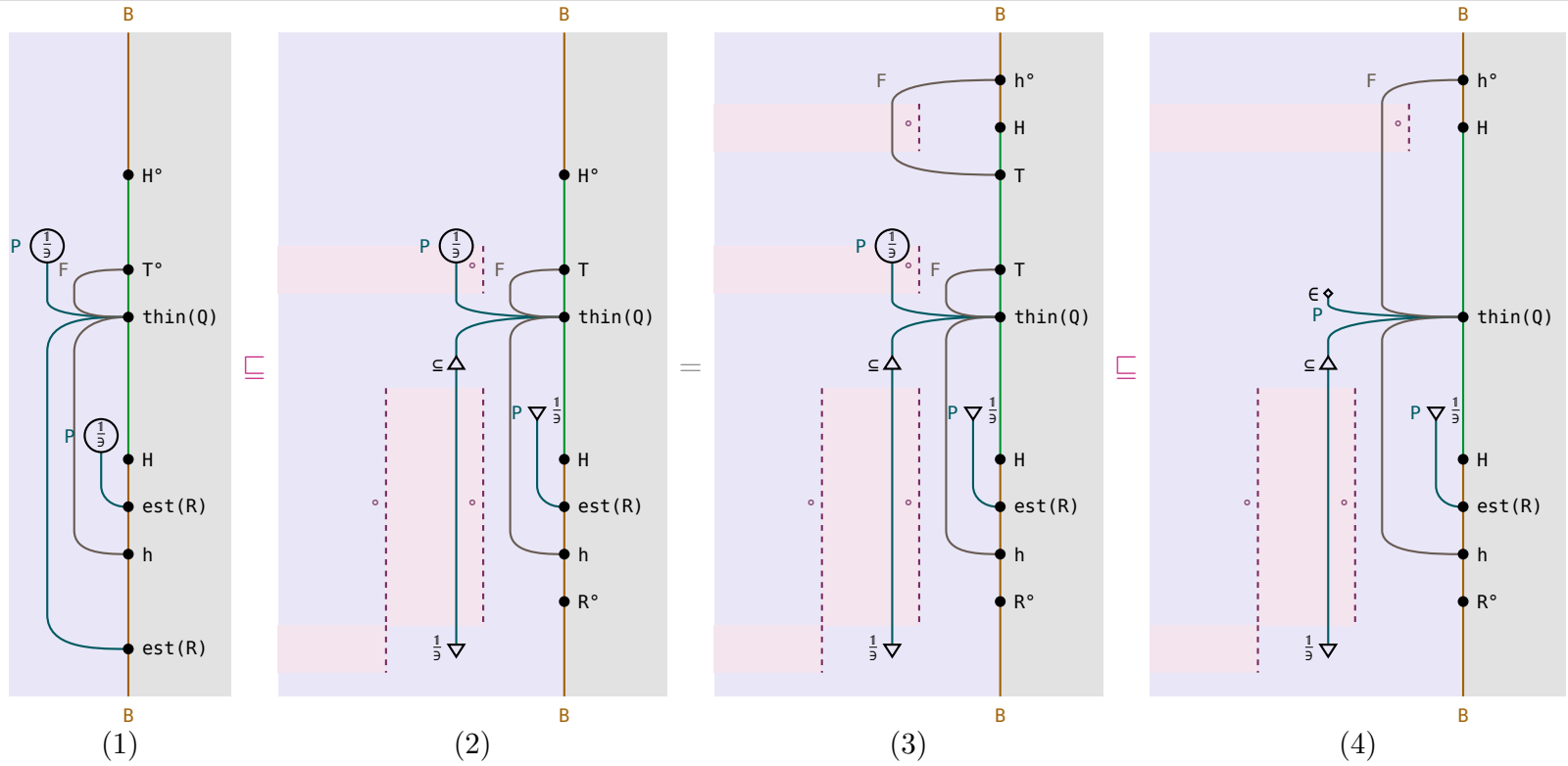
$$\frac{R}{\exists} \exists = R$$

$$\frac{S}{\exists} \text{est}(R) \sqsubseteq S$$

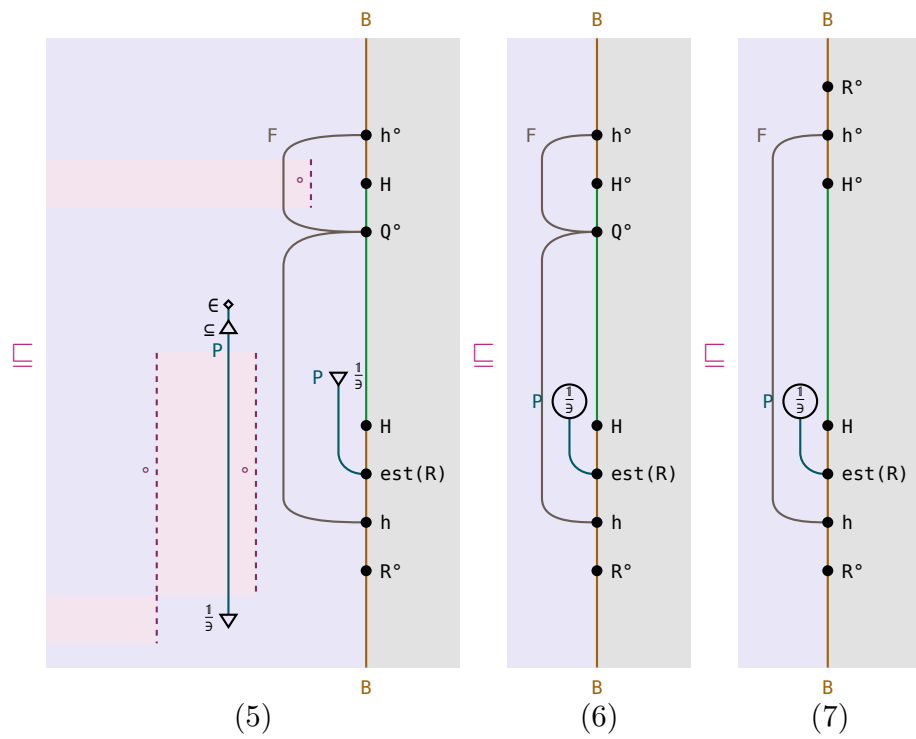
$$T^\circ F(H) h = H$$

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

H° followed by the body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input



1	$H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \epsilon \setminus (S R^\circ)$
2	$\sqsubseteq H^\circ \frac{T^\circ}{3} \text{thin}(Q) (\epsilon \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$H^\circ = h^\circ F(H^\circ) T$
3	$= h^\circ F(H^\circ) T \frac{T^\circ}{3} \text{thin}(Q) (\epsilon \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$T \frac{T^\circ}{3} \sqsubseteq \epsilon$
4	$\sqsubseteq h^\circ F(H^\circ) \epsilon \text{thin}(Q) (\epsilon \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$\epsilon \text{thin}(Q) \sqsubseteq Q^\circ \epsilon$



5	$\sqsubseteq h^\circ F(H^\circ) Q^\circ \epsilon (\epsilon \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$S (S \setminus R) \sqsubseteq R$
6	$\sqsubseteq h^\circ F(H^\circ) Q^\circ F(\frac{H}{3} \text{est}(R)) h R^\circ$	$\text{ThinCondition}(T, h, R, Q)$
7	$\sqsubseteq R^\circ h^\circ F(H^\circ) F(\frac{H}{3} \text{est}(R)) h R^\circ$	$\Rightarrow h^\circ F(H^\circ) Q^\circ \sqsubseteq R^\circ h^\circ F(H^\circ)$

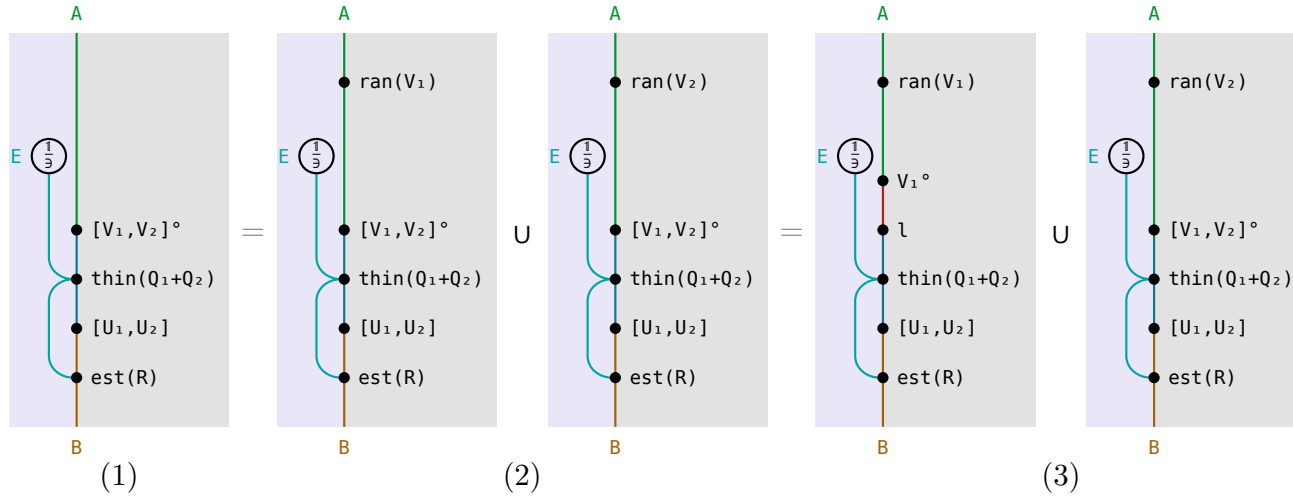
8	$= R^\circ \quad h^\circ \quad F(H^\circ \quad \frac{H}{\exists} \quad \text{est}(R)) \quad h \quad R^\circ$	$S^\circ \quad \frac{S}{\exists} \quad \text{est}(R) \quad \subseteq R^\circ$
9	$\subseteq R^\circ \quad h^\circ \quad F(R^\circ) \quad h \quad R^\circ$	$\text{Map}(f)$ $\implies f \text{ Pres } R \iff f^\circ F(R) \quad f \subseteq R$
10	$\subseteq R^\circ \quad R^\circ \quad R^\circ$	$R^\circ \quad R^\circ \subseteq R^\circ$
11	$\subseteq R^\circ \quad R^\circ$	$R^\circ \quad R^\circ \subseteq R^\circ$
12	$\subseteq R^\circ$	

§9.1.2. Checking the conditions

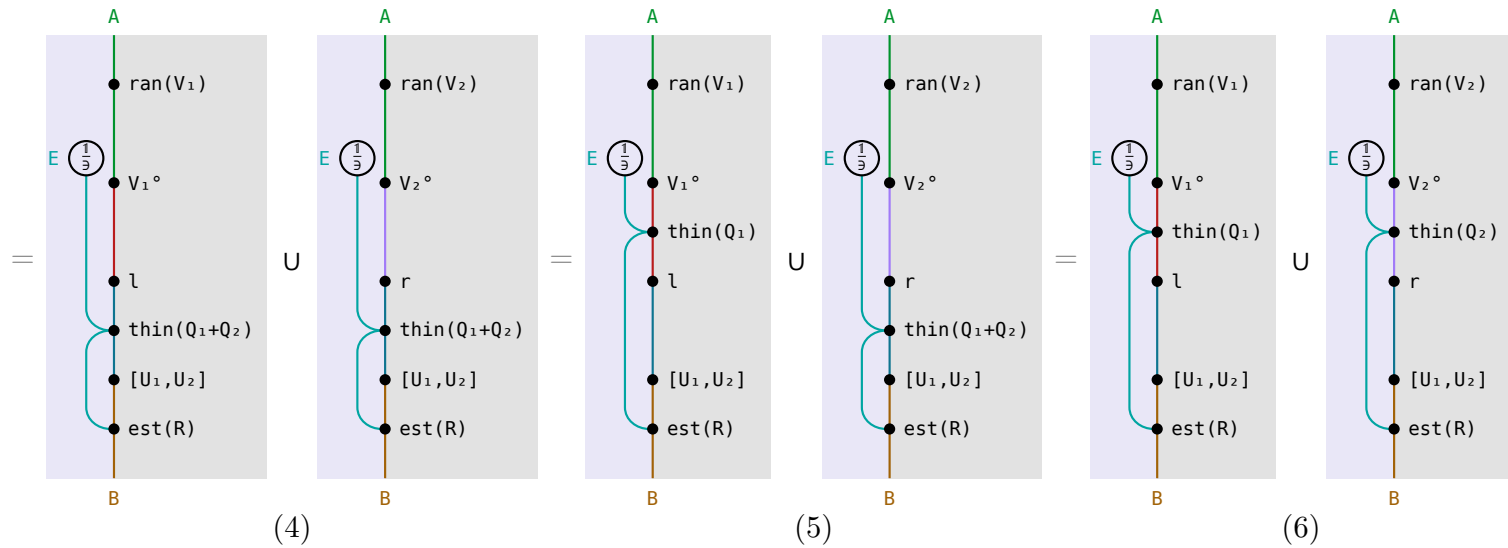
Proposition 9.1

$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$$

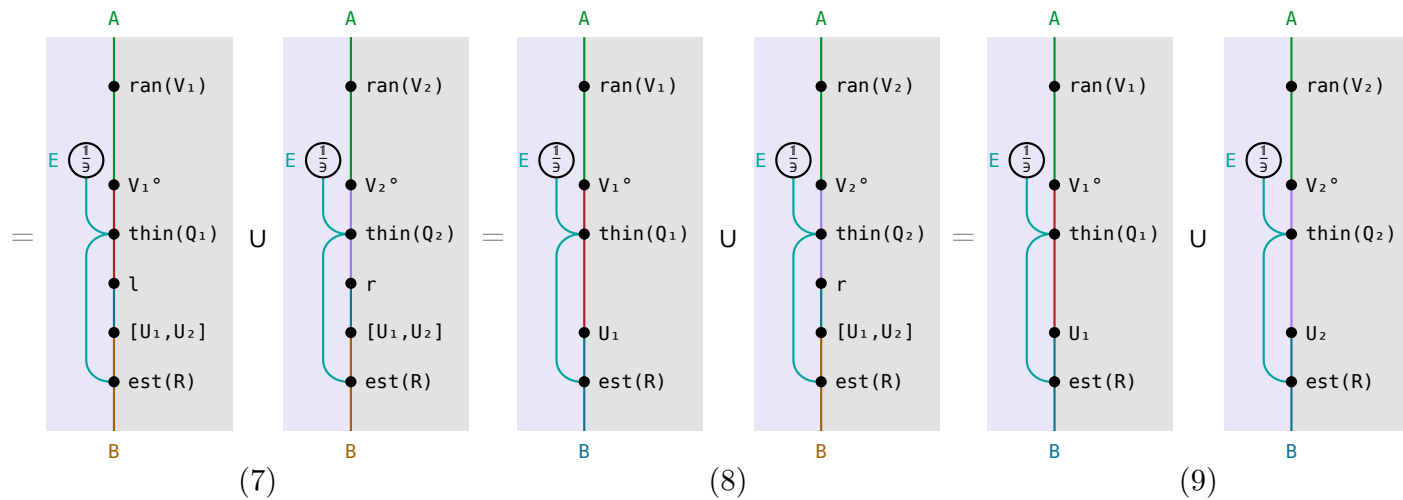
when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



1	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)$	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$
2	$= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l)$
3	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r)$



4	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(l) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) \ P(l)$
5	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(r) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_2) \ P(r)$
6	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	



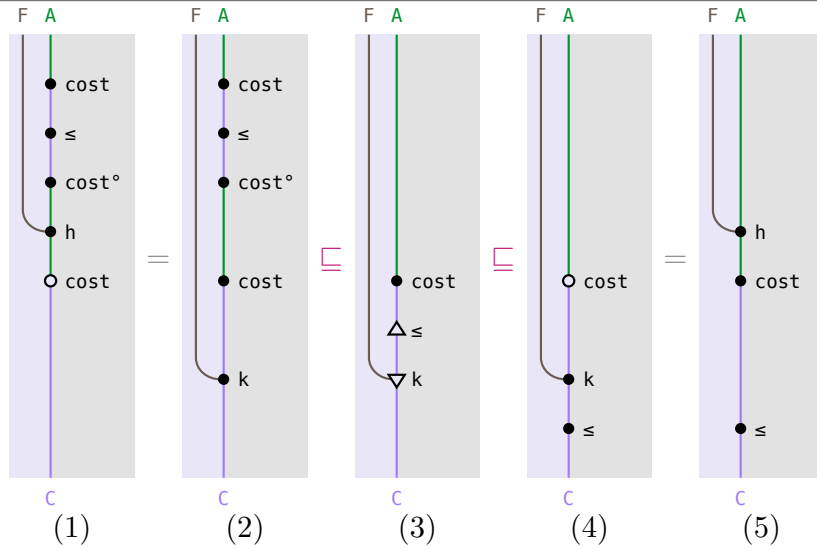
7	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$l \ [R, S] = R$
8	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$r \ [R, S] = S$
9	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$	

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$ Proposition 9.2

if h then **cost** equals **F(cost)** then a **k** monotonic on $\leq$, then **h** is monotonic on **cost $\leq$ cost $^\circ$** ,
which compares two values by comparing their **costs** under $\leq$

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h cost $\sqsubseteq$ h cost $\leq$

by shunting at the map **cost**, **F(cost $\leq$ cost $^\circ$)h $\sqsubseteq$ h cost $\leq$ cost $^\circ$** is **F(cost $\leq$ cost $^\circ$)h cost $\sqsubseteq$ h cost $\leq$** , which the chain proves



1	$F(\text{cost} \leq \text{cost}^\circ) \text{ h cost}$	$\text{h cost} = F(\text{cost}) \text{ k}$
2	$= F(\text{cost} \leq \text{cost}^\circ) F(\text{cost}) \text{ k}$	$\text{Map}(f) \wedge R = f S f^\circ$
3	$\sqsubseteq F(\text{cost}) F(\leq) \text{ k}$	$\implies R f \sqsubseteq f S$
4	$\sqsubseteq F(\text{cost}) \text{ k} \leq$	$F(\leq) \text{ k} \sqsubseteq \text{k} \leq$
5	$= \text{h cost} \leq$	$\text{h cost} = F(\text{cost}) \text{ k}$

(9.1.2a)

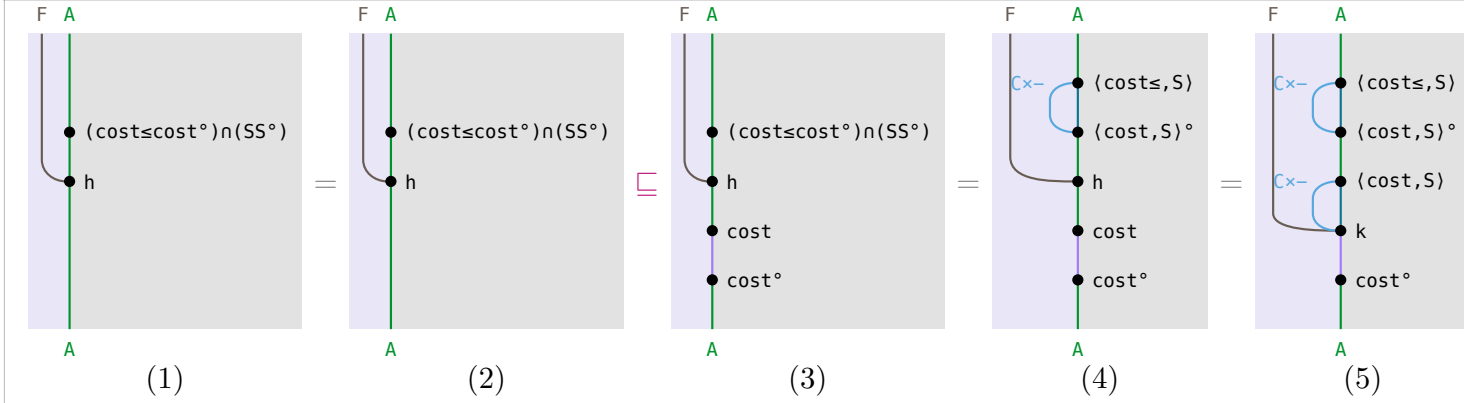
	the condition	how it is discharged
1	$F(R) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\text{cost}) \text{ k}$, $F(\leq) \text{ k} \sqsubseteq \text{k} \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
2	$F(R \text{ n } (H^\circ H)) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\langle \text{cost}, H^\circ \rangle) \text{ k}$, $F(\leq \times 1) \text{ k} \sqsubseteq \text{k} \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from
3	$QF(H) \text{ h} \sqsubseteq F(H) \text{ h} R$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R) \text{ h} \sqsubseteq \text{h} R$, $V H \sqsubseteq H R$	both conditions at once, split along the two arguments of a bifunctor

Map(cost) $\wedge$ Simple(S) $\wedge$ h cost = F((cost,S)) k $\wedge$ F($\leq \times 1$) k $\sqsubseteq$ k $\leq \Rightarrow$

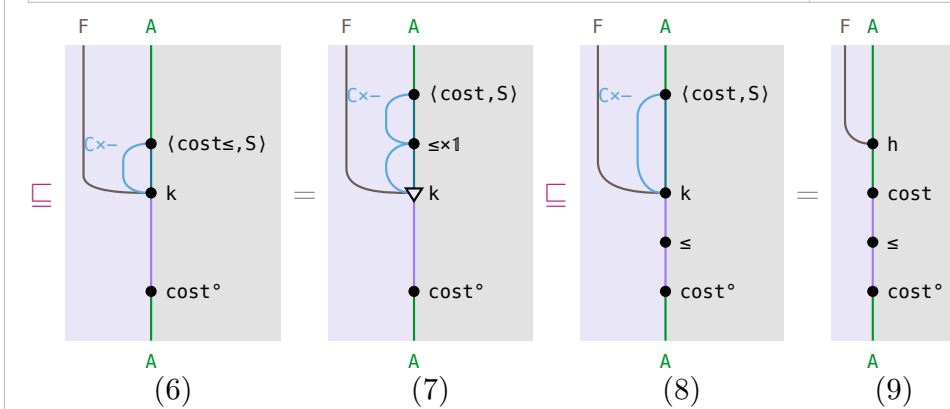
Proposition 9.3

F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$

improving each part by **R** within its **S**-context, then assembling by **h**, is below **hR**



1	F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h	
2	= F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h	Map(f)
3	$\sqsubseteq$ F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h cost cost $^\circ$	$\Rightarrow 1 \sqsubseteq f f^\circ$
4	= F((cost $\leq$, S) (cost, S) $^\circ$) h cost cost $^\circ$	$\langle X, Y \rangle (R, S)^\circ = (X R^\circ) \cap (Y S^\circ)$
5	= F((cost $\leq$, S) (cost, S) $^\circ$ (cost, S)) k cost $^\circ$	h cost = F((cost, S)) k

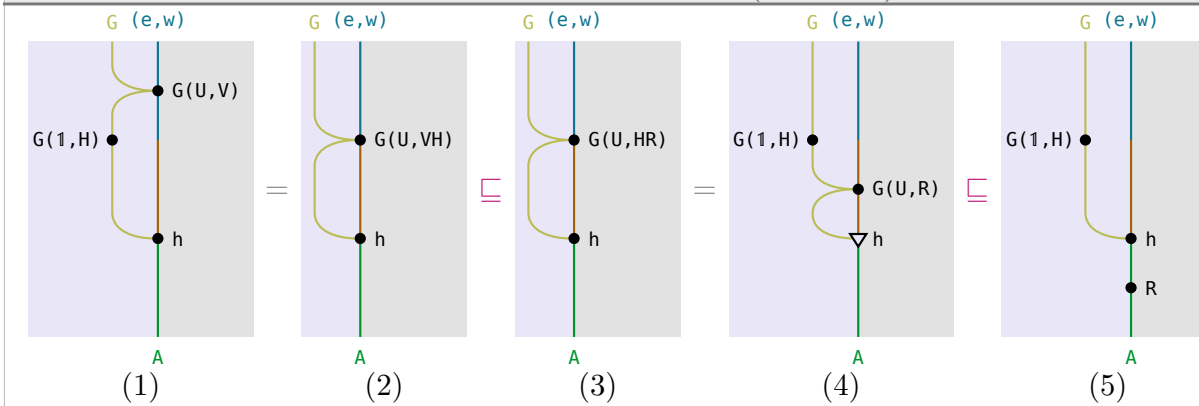


6	$\sqsubseteq$ F((cost $\leq$, S)) k cost $^\circ$	$\langle X, Y \rangle (R \times 1) = \langle X R, Y \rangle$
7	= F((cost, S)) F($\leq \times 1$) k cost $^\circ$	F($\leq \times 1$) k $\sqsubseteq$ k $\leq$
8	$\sqsubseteq$ F((cost, S)) k $\leq$ cost $^\circ$	h cost = F((cost, S)) k
9	= h cost $\leq$ cost $^\circ$	

G(U, R) h $\sqsubseteq$ h R $\wedge$ V H $\sqsubseteq$ H R $\Rightarrow$ G(U, V) G(1, H) h $\sqsubseteq$ G(1, H) h R

Proposition 9.4

thinning the parts by **U** in the first argument and by **V** in the second (G(U, V)), then solving by **H** and assembling by **h**, gives only what solving and assembling and then improving by **R** gives (G(1, H) h R)



1	G(U, V) G(1, H) h	
2	= G(U, V H) h	V H $\sqsubseteq$ H R
3	$\sqsubseteq$ G(U, H R) h	
4	= G(1, H) G(U, R) h	G(U, R) h $\sqsubseteq$ h R
5	$\sqsubseteq$ G(1, H) h R	

§9.2. The string edit problem

(9.2a)

	name	type	definition	meaning
1	Op	Type	$(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$	an edit operation copies, deletes or inserts one character
2			$FX = 1 + (Op \times X)$	$F(Op, X)$ is the empty sequence or one operation paired with an X
3	edit	$[Op] \rightarrow [Char] \times [Char]$	$\text{edit} = ([\text{base}, \text{step}])$, $\text{edit}([]) = ([], [])$, $\text{edit}([op] \# es) = \text{step}((op, \text{edit}(es)))$	the two strings an edit sequence turns one into the other
4			$\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$	the number of operations in a sequence
5	R	$[Op] \rightarrow [Op]$	$R = \text{length} \leq \text{length}^\circ$, $R(es, fs) \iff \text{length}(es) \leq \text{length}(fs)$	es is no longer than fs
6	V	$[Char] \times [Char] \rightarrow [Char] \times [Char]$	$V = \text{suffix}^\circ \times \text{suffix}^\circ$, $V(p, q) \iff \text{suffix}(\pi_1(p), \pi_1(q)) \wedge \text{suffix}(\pi_2(p), \pi_2(q))$	each string of the first pair is a suffix of the matching string of the second
7	Q	$F([Char] \times [Char]) \rightarrow F([Char] \times [Char])$	$l(val) \ Q \ l(val') \iff \text{true}$, $r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))$, $(\forall val. \forall val'. u = l(val) \implies v = l(val') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false}) \implies (u \ Q \ v \iff \text{false})$, $Q(l(u), l(u))$, $Q(r(p), r(q)) \iff V(\pi_2(p), \pi_2(q))$	the identity on the base summand; on the step summand any two operations, and the two pairs of strings by V
8	empty	$[Char] \times [Char] \rightarrow [Char] \times [Char]$	$\text{empty}(p, q) \iff p = q \wedge p = ([], [])$	holds only at the pair of two empty strings
9	unstep	$[Char] \times [Char] \rightarrow [Op] \times [Char] \times [Char]$	$q \in \text{unstep}(p) \implies \text{step}(q) = p$, $\text{unstep}([], []) = []$, $\text{unstep}([a] \# xs, []) = [(\text{del}(a), (xs, []))]$, $\text{unstep}([], [b] \# ys) = [(\text{ins}(b), ([], ys))]$, $\text{unstep}([a] \# xs, [b] \# ys) = \text{if } a = b \text{ then } [(cpy\ a, xs, ys)] \text{ else } [(\text{del}\ a, xs, [b] \# ys), (\text{ins}\ b, [a] \# xs, ys)]$	every split of a pair of strings into one operation and a smaller pair; step puts it back
10	[nil, cons]	$\text{Unit} + Op \times [Op] \rightarrow [Op]$	$[\text{nil}, \text{cons}](l(u)) = []$, $[\text{nil}, \text{cons}](r(o, es)) = [o] \# es$	[nil, cons]: the empty edit sequence, or one operation in front of a sequence
11	[base, step]	$\text{Unit} + Op \times [Char] \times [Char] \rightarrow [Char] \times [Char]$	$\text{base} = ([], [])$, $\text{step}((cpy(a), (xs, ys))) = ([a] \# xs, [a] \# ys)$, $\text{step}((\text{del}(a), (xs, ys))) = ([a] \# xs, ys)$, $\text{step}((\text{ins}(a), (xs, ys))) = (xs, [a] \# ys)$	[base, step]: the two empty strings, or one operation added to the front of the pair

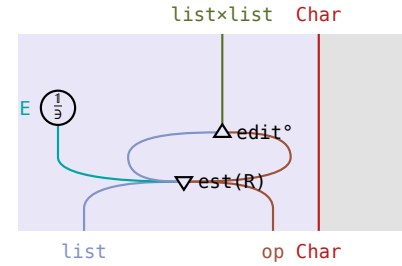
$$\text{mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R)), (\mu X : \frac{[\text{base}, \text{step}]^\circ}{3} \text{thin}(Q) \text{P}(F(X) [\text{nil}, \text{cons}]) \text{est}(R)) \sqsubseteq \frac{\text{edit}^\circ}{3} \text{est}(R) \quad (9.2b)$$

a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

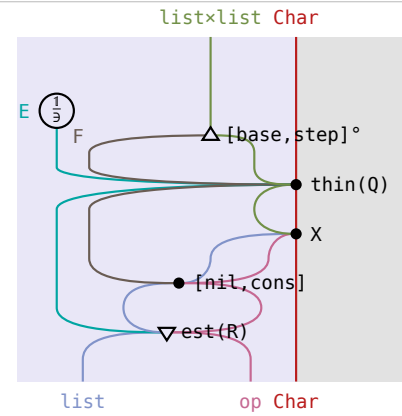
circuit

$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}(\text{edit}^\circ)} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$
the specification — $\text{edit}([\text{ }]) = ([\text{ }], [\text{ }])$, $\text{edit}([op] \# es) = \text{step}((op, \text{edit}(es)))$

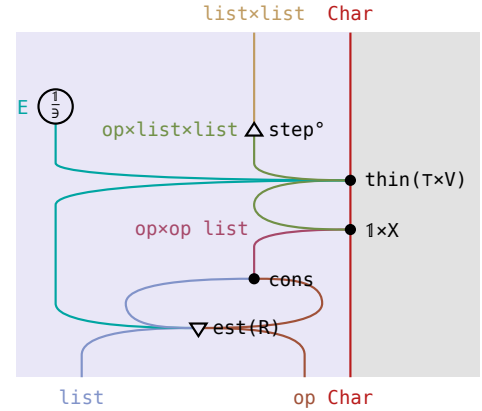
Hinze–Marsden



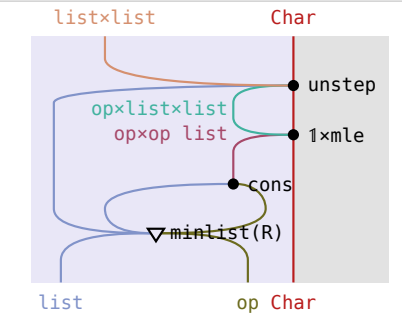
$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}([\text{base}, \text{step}]^\circ)} \text{thin}(Q) \text{P}[\text{Char}]^2 \xrightarrow{\text{P}(F(X) [\text{nil}, \text{cons}])} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$
at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is Proposition 9.2 at $\text{length} \triangleq ([\text{zero}, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R . The thinning condition $\sqsubseteq QF(1, \text{edit}^\circ) \alpha \sqsubseteq F(1, \text{edit}^\circ) \alpha R$ over $F(0p, [\text{Char}] \times [\text{Char}])$ is Proposition 9.4 at $U \triangleq T$ — $F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, — $\text{edit}(1 \times \text{suffix}) \sqsubseteq R^\circ \text{edit}$, $\text{edit}(\text{suffix} \times 1) \sqsubseteq R^\circ \text{edit}$. $\text{edit} \cdot \text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del , never lengthening the sequence



$[\text{Char}] \xrightarrow{\frac{1}{3}} \text{P}[\text{Char}]^2 \xrightarrow{\text{E}(\text{step}^\circ)} \text{thin}(T \times V) \text{P}(0p \times [\text{Char}] \times [\text{Char}]) \xrightarrow{\text{P}((1 \times X) \text{cons})} \text{P}[0p] \xrightarrow{\text{est}(R)} [0p]$
= Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty , the coreflexive on (xs, ys) with both lists empty, is where base returns



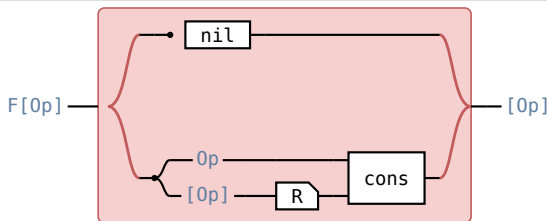
$[\text{Char}] \xrightarrow{\text{unstep}} [0p \times [\text{Char}] \times [\text{Char}]] \xrightarrow{\text{list}((1 \times \text{mle}) \text{cons})} [[0p]] \xrightarrow{\text{minlist}(R)} [0p]$
 unstep implements $\frac{\text{step}^\circ}{3} \text{thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and $\text{minlist}(R)$ implements $\text{est}(R)$. The same subproblem is solved many times over, so the running time is exponential in the two lengths



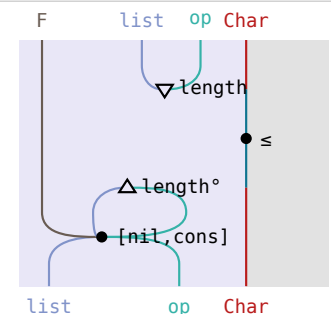
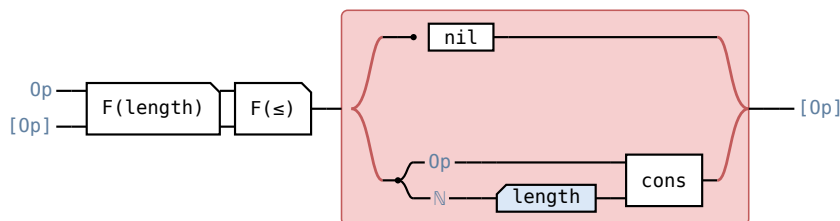
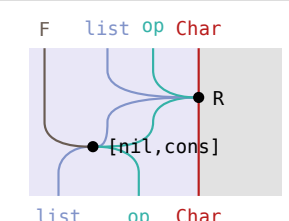
$$F(R) [\text{nil}, \text{cons}] \sqsubseteq [\text{nil}, \text{cons}] R \quad (9.2c)$$

comparing the tails by R and then putting an operation in front ($F(R) \alpha$) relates only sequences that R also relates once the operation is in front (αR)

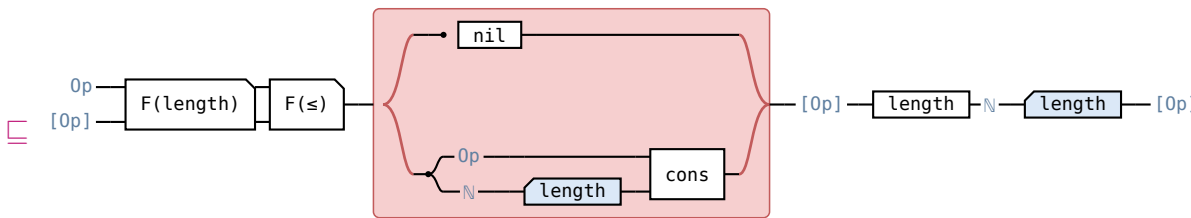
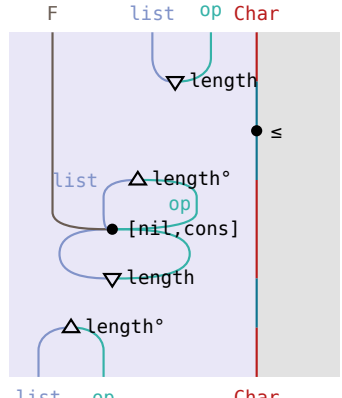
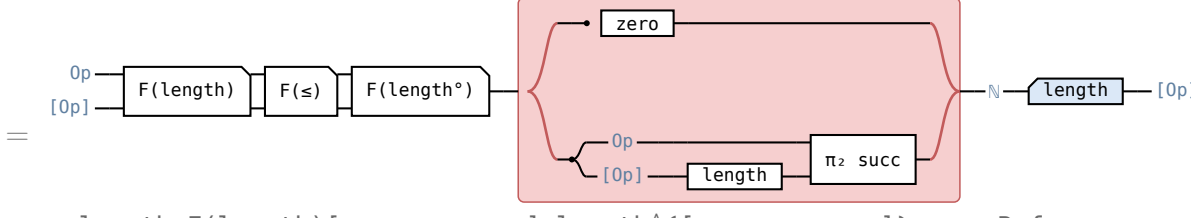
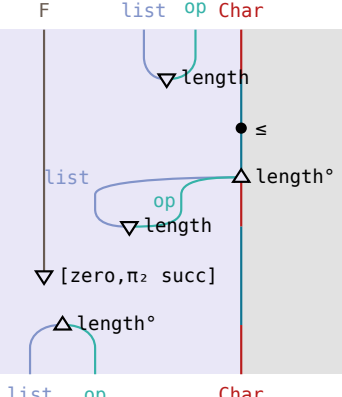
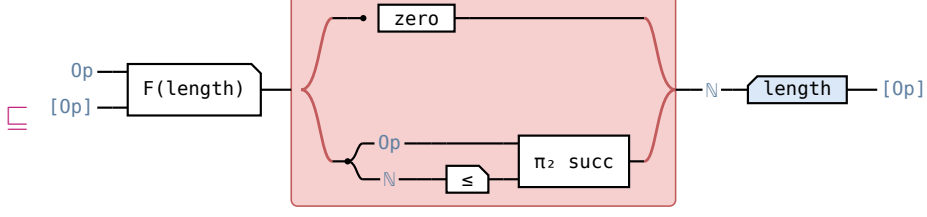
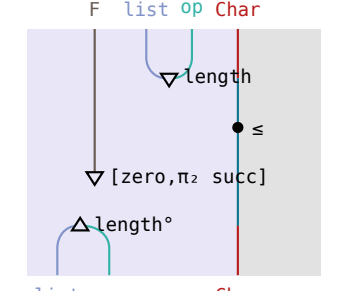
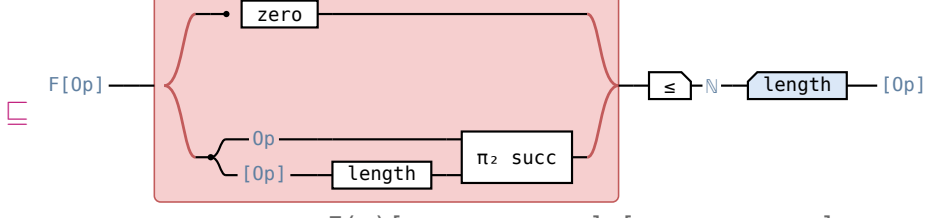
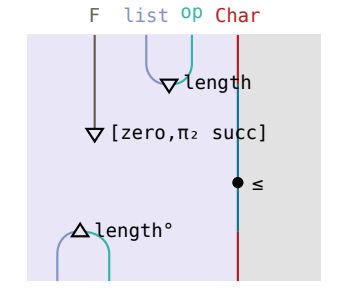
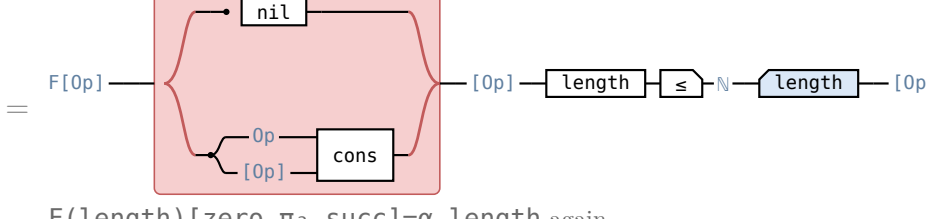
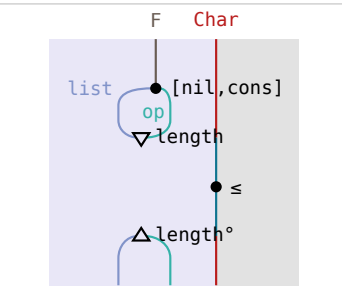
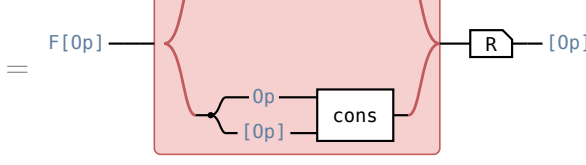
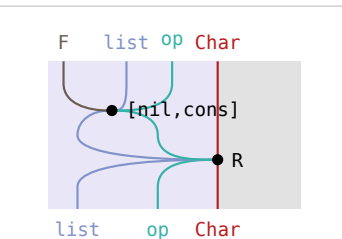
circuit



Hinze–Marsden

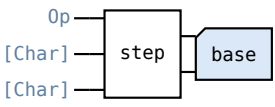
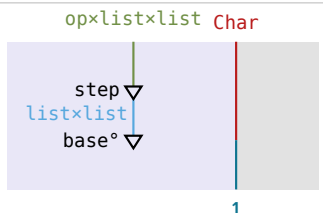
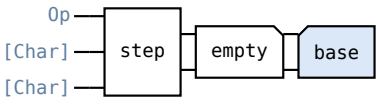
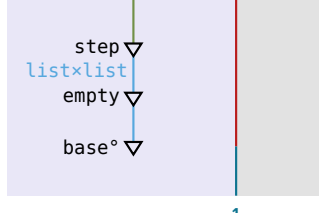
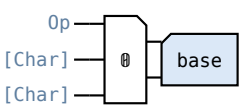
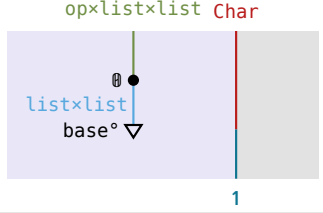
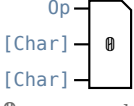
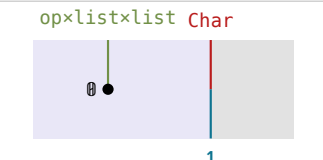


$R \triangleq \text{length} \leq \text{length}^\circ$ — $es \ R \ fs \iff \text{length}(es) \leq \text{length}(fs)$; F preserves composition

circuit	Hinze–Marsden
 length is a map, so entire: $1 \sqsubseteq \text{length} \text{ length}^\circ$	
 $\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length} \triangleq ([\text{zero}, \pi_2 \text{ succ}]) - \text{es } R \text{ fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})$	
 length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
 succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
 $F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
 $R \triangleq \text{length} \leq \text{length}^\circ - \text{es } R \text{ fs} \iff \text{length}(\text{es}) \leq \text{length}(\text{fs})$	

$Q \vdash F(\text{edit}^\circ) [\text{nil}, \text{cons}] \sqsubseteq F(\text{edit}^\circ) [\text{nil}, \text{cons}] R$ (9.2d)	
every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq \tau - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}$, $r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))$, $(\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})$ $\implies (u \ Q \ v \iff \text{false})$; and the bifunctor F preserves composition: $F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{ edit}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $V \text{ edit}^\circ \sqsubseteq \text{edit}^\circ R - \text{edit} \ V^\circ \sqsubseteq R^\circ \text{ edit}$	
$=$ F preserves composition: $F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(\tau, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: cons adds one to both lengths whatever the two operations are	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2e) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
$=$ $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	
$\sqsubseteq$ $\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins . Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	
$\sqsubseteq$ $\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	
$\sqsubseteq$ R° is transitive, $\leq$ being so	

$\text{step } \text{base}^\circ = \emptyset$ (9.2f) no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

§9.2.1. Tabulation

(9.2.1a)

1	$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $\text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)])$, $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left
2	$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $\text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)] \triangleright (xus), xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right
3	$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $\text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§9.3. Optimal bracketing

(9.3a)

	name	type	definition	meaning
1	tree(A)	Type	$\text{tree}(A) \triangleq \text{tip}(A) \mid \text{bin}(\text{tree}(A), \text{tree}(A))$	a tree is a tip holding one value, or a node joining two trees; $\mathbf{h} \triangleq [\text{tip}, \text{bin}]$
2	flatten	$\text{tree}(A) \rightarrow \text{LA}$	$\langle [\text{wrap}, \text{cat}] \rangle = \text{flatten}, \text{flatten}(\text{tip}(a)) = [a],$ $\text{flatten}(\text{bin}(xs, ys)) = \text{cat}(\text{flatten}(xs), \text{flatten}(ys))$	the tips of a tree, left to right; $\mathbf{H} = \text{flatten}^\circ$ builds every tree over a list
3	opb(p)	$\text{Int} \times \text{S}$	$\text{opb}(p) \triangleq ((\text{cb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))) + \pi_1(\pi_1(p))) + \pi_1(\pi_2(p)), \text{sb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))))$	joining two subtrees costs cb of their sizes plus the cost of each
4	$\langle \text{cost}, \text{size} \rangle$	$\text{tree}(A) \rightarrow \text{Int} \times \text{S}$	$\langle \text{cost}, \text{size} \rangle(\text{tip}(a)) = (0, \text{st}(a)),$ $\langle \text{cost}, \text{size} \rangle(\text{bin}(xs, ys)) = \text{opb}(\langle \text{cost}, \text{size} \rangle(xs), \langle \text{cost}, \text{size} \rangle(ys))$	the cost of evaluating a bracketing, paired with the size of its result
5	sz	$\text{LA} \rightarrow \text{S}$	$\text{sz}([a]) = \text{st}(a), \text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs)),$ $\text{Assoc}(\text{sb}) \implies \pi_2(\langle \text{cost}, \text{size} \rangle(xs)) = \text{sz}(\text{flatten}(xs))$	sb associative, so the size depends only on the flattened list
6	R	$\text{tree}(A) \rightarrow \text{tree}(A)$	$R = \text{cost} \leq \text{cost}^\circ, R(xs, ys) \iff \text{cost}(xs) \leq \text{cost}(ys)$	t costs no more than t'
7	g	$\text{F}(\text{Int} \times \text{LA}) \rightarrow \text{Int}$	$g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1], g(\text{l}(a)) = 0,$ $g(\text{r}((cx, xs), cy, ys)) = (\text{cb}(\text{sz}(xs), \text{sz}(ys)) + cx) + cy$	the cost of the top step alone: zero at a tip, cb of the two sizes at a node
8	nonsingle	$\text{LA} \rightarrow \text{LA}$	$xs \text{ nonsingle } ys \iff xs = ys \wedge (\exists a. \exists zs. xs = [a] \# zs)$	holds at the lists of two or more elements
9	splits	$\text{LA} \rightarrow [\text{L } A \times \text{L } A]$	$\text{splits} = (\text{inits}^+, \text{tails}^+) \text{ zip}, \text{splits}(xs) = \text{zip}(((\text{inits}^+)(xs), (\text{tails}^+)(xs)))$	every way to cut a list into two non-empty parts — an implementation of $\frac{\text{cat}^\circ}{\exists}$
10	array	$\text{LA} \rightarrow [\text{tree } A]$	$\text{array} \triangleq \text{inits list}(\text{row}), \text{array}(xs, xss) \iff xss = \text{list}(\text{row}, \text{inits}(xs))$	the best trees of every segment, held as rows
11	row	$\text{LA} \rightarrow [\text{tree } A]$	$\text{row} = \text{tails list}(\text{mct}), \text{row}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{tails}(xs))$	the best tree of every non-empty suffix
12	col	$\text{LA} \rightarrow [\text{tree } A]$	$\text{col} = \text{inits list}(\text{mct}), \text{col}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{inits}(xs))$	the best tree of every non-empty prefix
13	mix	$[\text{tree } A] \times [\text{tree } A] \rightarrow \text{tree}(A)$	$\text{mix} = \text{zip list}(\text{bin}) \text{ minlist}(R), \text{mix}(p, xs) \iff xs = \text{minlist}(R) (\text{list bin zip})$	join the matching prefix and suffix trees and keep a cheapest
14	next	$[\text{tree } A] \times [\text{tree } A] \rightarrow [\text{tree } A]$	$\text{next} = (\pi_1, \text{mix}) \text{ snoc}, \text{next}(p, xss) \iff xss = \text{snoc}((\pi_1(p), \text{minlist}(R) (\text{list bin zip})))$	add the next best tree to the end of the column

15

name	type	definition	meaning
process	$A \times [[\text{tree } A]] \rightarrow [\text{tree } A]$	$\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next}) ,$ $\text{process}((a, xsss), xss) \Leftrightarrow$ $\text{loop}(\text{next})(([\text{tip } a], xsss), xss)$	start the column at a tip and run next over the rows

Assoc(sb) $\Rightarrow$ F((cost, flatten)) g = [tip, bin] cost (9.5) pairing each subtree with its cost and its flattening and then applying g gives what building the tree by [tip, bin] and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow \boxed{F((\text{cost}, \text{flatten}))} \rightarrow F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{LA} \rightarrow \text{sz} \end{array} \right\} \\ \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{LA} \rightarrow \text{sz} \end{array} \right\} \end{array} \right\} \rightarrow \boxed{\text{opb}} \rightarrow \text{Int}$ definition of g; coproducts and products — $g \triangleq [\multimap 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \\ \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \end{array} \right\} \rightarrow \boxed{\text{opb}} \rightarrow \text{Int}$ flatten sz=size, since sb is associative — $\text{sz}([a]) = \text{st}(a)$, $\text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs))$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \rightarrow \text{Int} \\ \text{size} \rightarrow S \end{array} \right\} \end{array} \right\} \rightarrow \text{Int}$ $(\text{cost}, \text{size}) \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = (\text{cost}, \text{size})^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \boxed{\text{tip}} \rightarrow \text{tree}(A) \rightarrow \text{cost} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \text{cost} \end{array} \right\} \rightarrow \text{Int}$ tip cost=zero; products, $(\text{cost}, \text{size}) \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \boxed{\text{tip}} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \end{array} \right\} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{ cost}$	

$F(\leq \times 1) \ g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	

$$\text{mct}=(\text{single}\rightarrow\text{head tip},\langle\text{init col},\text{tail row}\rangle\text{mix}), \text{ Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap},\text{cat}]^\circ}{\mathfrak{g}} \text{P}([\text{tip},(X \times X) \text{bin}]) \text{est}(\text{R})) \sqsubseteq \frac{\text{flatten}^\circ}{\mathfrak{g}} \text{est}(\text{R}) \quad (9.3b)$$

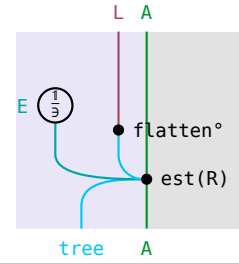
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{flatten}^\circ) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

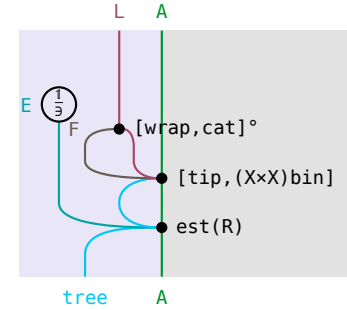
the specification — $\text{flatten}(\text{tip}(\text{a})) = [\text{a}]$, $\text{flatten}(\text{bin}(\text{xs},\text{ys})) = \text{cat}(\text{flatten}(\text{xs}),\text{flatten}(\text{ys}))$

Hinze–Marsden



$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}([\text{wrap},\text{cat}]^\circ) \rightarrow \text{PF}(\text{LA}) \rightarrow \text{P}([\text{tip},(X \times X)\text{bin}]) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,
 $\sqsubseteq \text{F}(\text{Rn}(\text{flatten} \text{ flatten}^\circ)) \text{h} \sqsubseteq \text{hR}$ — only trees with the same flattening are compared — which is Proposition 9.3 at $\text{H}^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip},\text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) \text{g}$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) \text{g} \sqsubseteq \text{g} \leq (\text{g} \text{ monotonic on } \leq \text{ in its two cost arguments})$

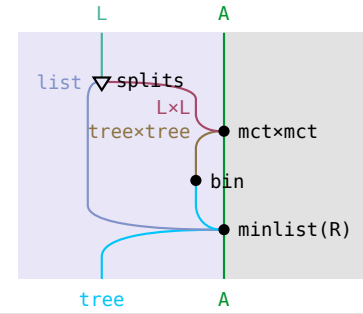


$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{cat}^\circ) \rightarrow \text{P}(\text{LA}^2) \rightarrow \text{P}((X \times X)\text{bin}) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

= Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{LA} \rightarrow \text{splits} \rightarrow [\text{LA}^2] \rightarrow \text{list}((\text{mct} \times \text{mct})\text{bin}) \rightarrow [\text{tree}(\text{A})] \rightarrow \text{minlist}(\text{R}) \rightarrow \text{tree}(\text{A})$$

$\sqsubseteq \text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{\mathfrak{g}}$ and $\text{minlist}(\text{R})$ implements $\text{est}(\text{R})$. Exponential, since the segments of one list overlap



$$\text{mct}=(\text{single}\rightarrow\text{head tip},\langle\text{init col},\text{tail row}\rangle\text{mix}) \quad (9.7)$$

= the tabulation: $\text{mct } \text{xs}$ is needed for every non-empty segment xs , so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(\text{R})$

$$\text{col}=(\text{single}\rightarrow\text{head tip wrap},\langle\text{init col},\text{tail row}\rangle\text{next}) \quad (9.8)$$

$$\text{cons col}=(1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row}=(\text{single}\rightarrow\text{head tip wrap},\langle\text{mct},\text{tail row}\rangle\text{cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq \langle \pi_1, \text{mix} \rangle \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

$$\text{array}=(\text{fstcol},\text{addcol}), \text{fstcol} \triangleq \text{tip wrap wrap}$$

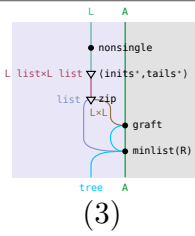
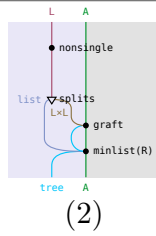
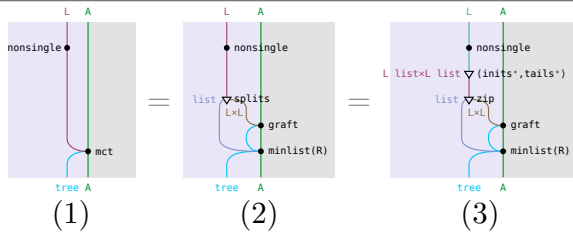
$$\text{addcol} \triangleq \langle \pi_1 \text{ tip wrap}, \text{step} \rangle \text{ cons}, \text{step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

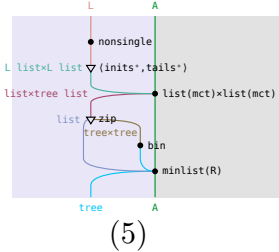
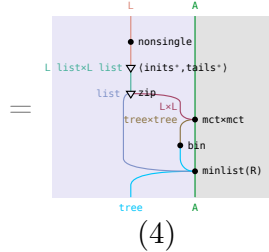
nonsingle mct = nonsingle (init col,tail row) mix

(9.7)

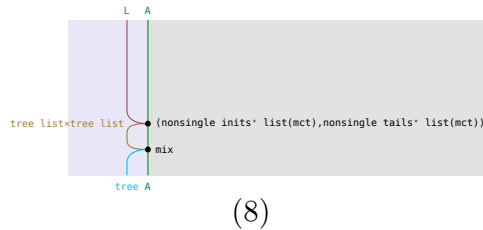
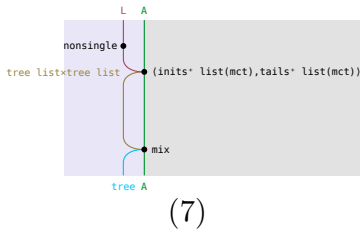
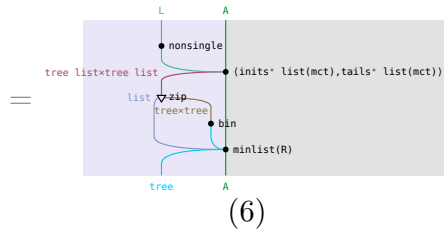
on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



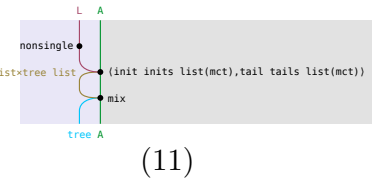
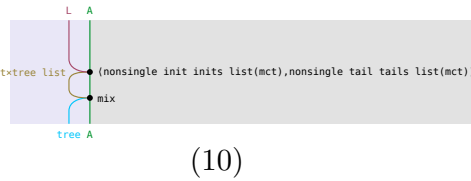
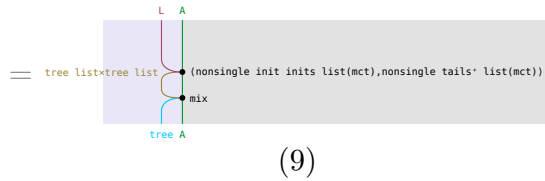
1	nonsingle mct	nonsingle splits list(graft) minlist(R)
2	= nonsingle splits list(graft) minlist(R)	splits = (inits ⁺ , tails ⁺) zip
3	= nonsingle (inits ⁺ , tails ⁺) zip list(graft) minlist(R)	graft $\triangleq$ (mct $\times$ mct) bin



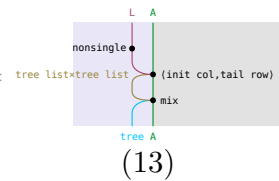
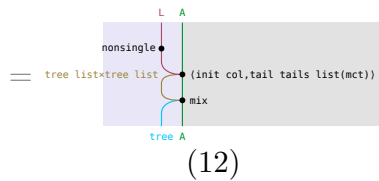
4	= nonsingle (inits ⁺ , tails ⁺) zip list(mct $\times$ mct) list(bin) minlist(R)	zip list(f $\times$ g) M = (list(f) $\times$ list(g)) zip M
5	= nonsingle (inits ⁺ , tails ⁺) (list(mct) $\times$ list(mct)) zip list(bin) minlist(R)	$\langle R, S \rangle (U \times V) = \langle R U, S V \rangle$



6	= nonsingle (inits ⁺ list(mct), tails ⁺ list(mct)) zip list(bin) minlist(R)	mix = zip list(bin) minlist(R)
7	= nonsingle (inits ⁺ list(mct), tails ⁺ list(mct)) mix	nonsingle $\langle U, V \rangle = \langle \text{nonsingle } U, \text{nonsingle } V \rangle$
8	= (nonsingle inits ⁺ list(mct), nonsingle tails ⁺ list(mct)) mix	nonsingle inits ⁺ M = nonsingle init inits M



9	= (nonsingle init inits list(mct), nonsingle tails ⁺ list(mct)) mix	nonsingle tails ⁺ M = nonsingle tail tails M
10	= (nonsingle init inits list(mct), nonsingle tail tails list(mct)) mix	nonsingle $\langle U, V \rangle = \langle \text{nonsingle } U, \text{nonsingle } V \rangle$
11	= nonsingle (init inits list(mct), tail tails list(mct)) mix	col = inits list(mct)

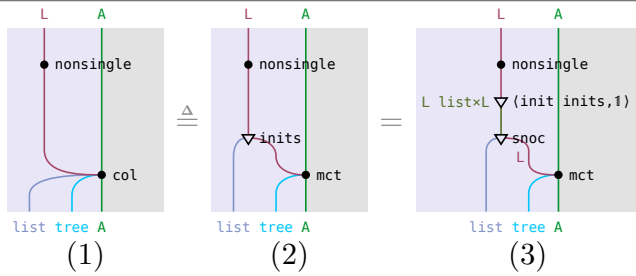


12	= nonsingle (init col, tail tails list(mct)) mix	row = tails list(mct)
13	= nonsingle (init col, tail row) mix	

nonsingle col = nonsingle (init col,tail row) next

(9.8)

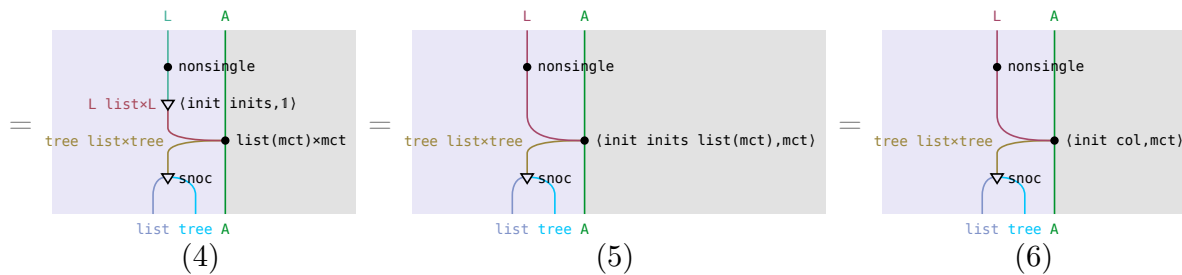
on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**



(2)

(3)

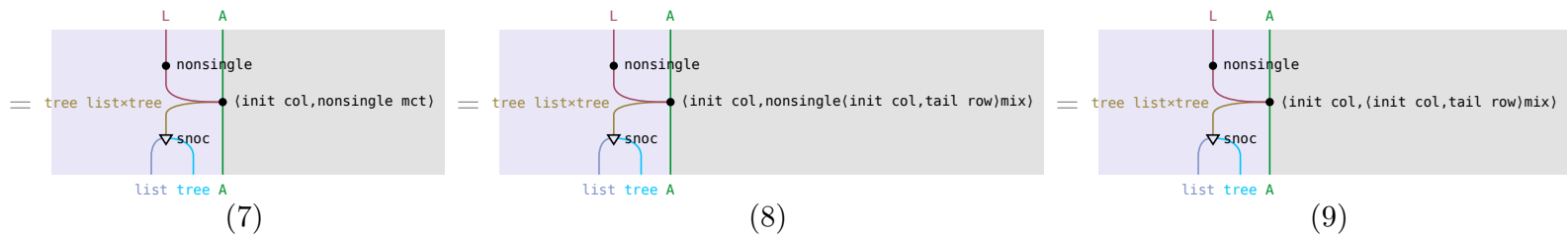
1	nonsingle col	col $\triangleq$ inits list(mct)
2	$\triangleq$ nonsingle inits list(mct)	nonsingle inits M = nonsingle (init inits,1) snoc M
3	= nonsingle (init inits,1) snoc list(mct)	snoc list(f) = (list(f) $\times$ f) snoc



(5)

(6)

4	= nonsingle (init inits,1) (list(mct) $\times$ mct) snoc	$\langle R,S \rangle (U \times V) = \langle R U, S V \rangle$
5	= nonsingle (init inits list(mct),mct) snoc	col = inits list(mct)
6	= nonsingle (init col,mct) snoc	nonsingle $\langle U,V \rangle =$ nonsingle $\langle U,$ nonsingle $V \rangle$



(8)

(9)

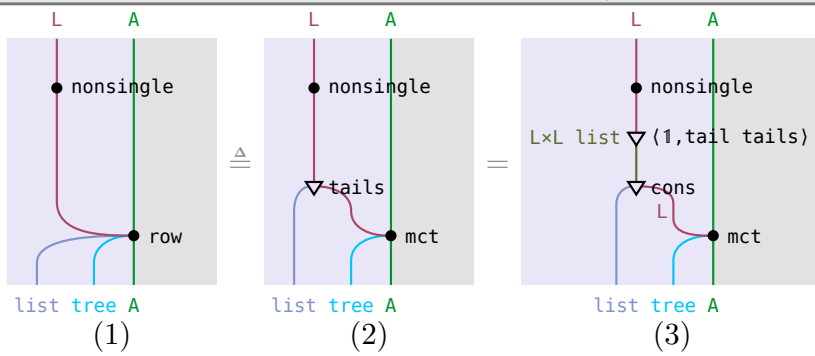
7	= nonsingle (init col,nonsingle mct) snoc	(9.7) nonsingle mct = nonsingle (init col,tail row) mix
8	= nonsingle (init col,nonsingle (init col,tail row) mix) snoc	nonsingle $\langle U,V \rangle =$ nonsingle $\langle U,$ nonsingle $V \rangle$
9	= nonsingle (init col,(init col,tail row) mix) snoc	$\langle \text{init col}, V \rangle \langle \pi_1, M \rangle = \langle \text{init col}, \langle \text{init col}, V \rangle M \rangle$



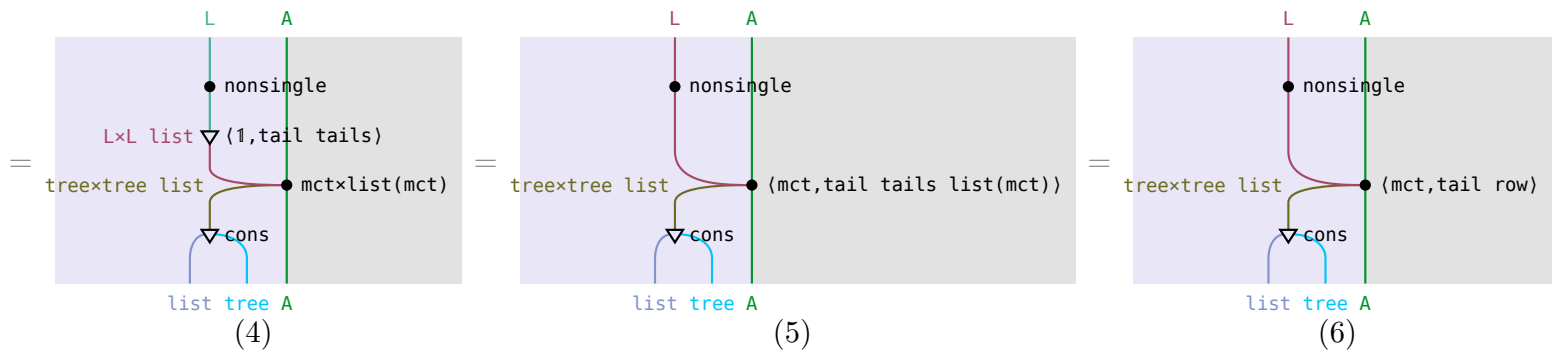
(11)

10	= nonsingle (init col,tail row) $\langle \pi_1, \text{mix} \rangle$ snoc	next = $\langle \pi_1, \text{mix} \rangle$ snoc
11	= nonsingle (init col,tail row) next	

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**



1	nonsingle row	row $\triangleq$ tails list(mct)
2	$\triangleq$ nonsingle tails list(mct)	nonsingle tails M = nonsingle (1,tail tails) cons M
3	= nonsingle (1,tail tails) cons list(mct)	cons list(f) = (f $\times$ list(f)) cons

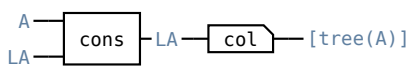


4	= nonsingle (1,tail tails) (mct × list(mct)) cons	(R,S) (U × V) = (R U,S V)
5	= nonsingle (mct,tail tails list(mct)) cons	row = tails list(mct)
6	= nonsingle (mct,tail row) cons	

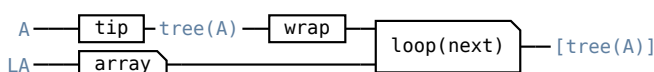
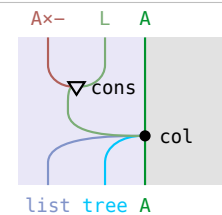
```
cons col = (1 x array) process
```

the column of **a** consed onto **x** is what **process** makes of **a** and the array of **x**

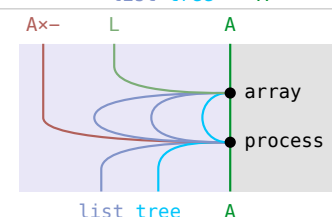
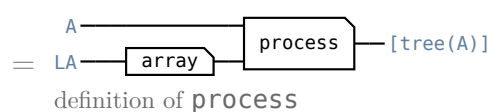
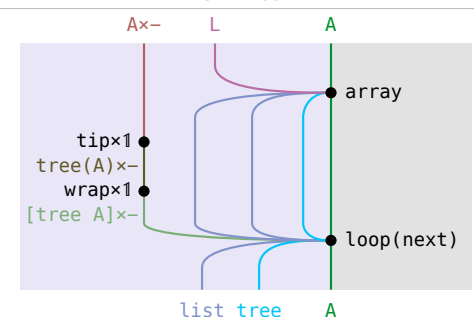
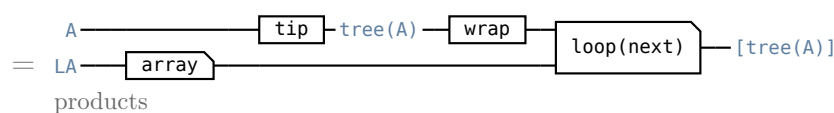
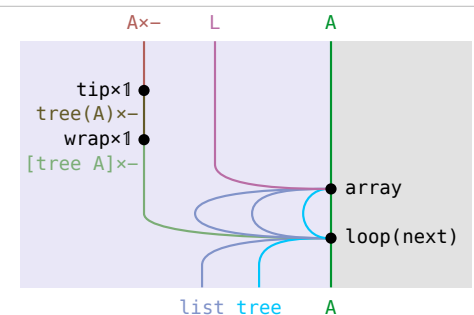
circuit

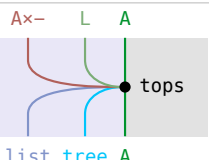
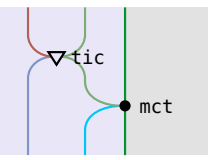
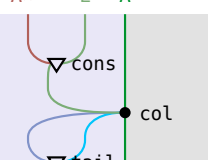
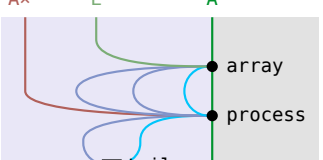


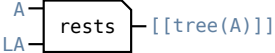
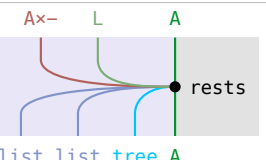
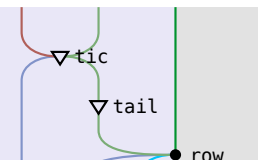
Hinze–Marsden



= Exercise 9.13 with `kcons col, gtip wrap, hrow, fnext`: its two equations are (9.8) — (9.8) `nonsingle col = nonsingle (init col, tail row) next` — at `a:[b]` and at `a:(u++[b])`, with `[a] col=[tip(a)]`



tops = (1 × array) process tail (9.3c) the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(mct)} \text{tree(A)}$ definition of tops$\triangleq$tic list(mct), our name	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{col} \text{tail} \text{tree(A)}$ tail list(f)=list(f) tail; definition of col	
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{process} \text{tail} \text{tree(A)}$ (9.9) — (9.9) cons col = (1 × array) process	

rests = π_2 array (9.3d) the rest of each new row is the array of x	
circuit	Hinze–Marsden
	
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(tail row)} \text{tree(A)}$ definition of rests$\triangleq$tic list(tail row), our name	
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{tree(A)}$ tic list(tail)=π_2 inits; definition of array	

$\text{newrows} = (1 \times \text{array}) \text{ step}$ (9.3e)	
the rows of the array of $\mathbf{a}:\mathbf{x}$ below its first are what step makes of $\mathbf{a}$ and the array of $\mathbf{x}$	
circuit	Hinze–Marsden
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{row}) \rightarrow [[\text{tree}(\text{A})]]$ definition of $\text{newrows} \triangleq \text{tic list}(\text{row})$, our name	
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{mct, tail row cons}) \rightarrow [[\text{tree}(\text{A})]]$ (9.10) — (9.10) nonsingle row = nonsingle (mct, tail row) cons — on the non-singleton tic lists	
$= \text{LA} \rightarrow \begin{matrix} \text{tops} \\ \text{rests} \end{matrix} \rightarrow \text{zip} \rightarrow [\text{tree}(\text{A}) \times [\text{tree}(\text{A})]] \rightarrow \text{list}(\text{cons}) \rightarrow [[\text{tree}(\text{A})]]$ $\text{list}(f,g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{LA} \rightarrow \text{array} \rightarrow \text{step} \rightarrow [[\text{tree}(\text{A})]]$ $\text{tops} = (1 \times \text{array})$ process tail and rests = π_2 array; products; definition of step	

$\text{cons array} = (1 \times \text{array}) \text{ addcol}$ (9.3f)	
the array of a consed onto x is what addcol makes of a and the array of x	
circuit	Hinze–Marsden
$=$ definition of array	
$=$ $\text{cons inits} = (\pi_1 \text{ wrap, tic}) \text{ cons}$, abbreviating cons inits tail by tic	
$=$ $\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons}$; definition of newrows	
$=$ $\text{wrap row} = \text{tip wrap}$	
$=$ $\text{newrows} = (1 \times \text{array}) \text{ step}$; products; definition of addcol	

§9.4. Data compression

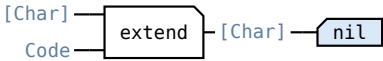
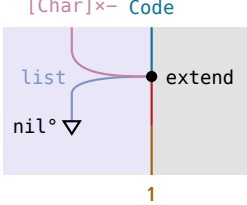
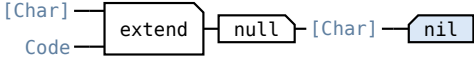
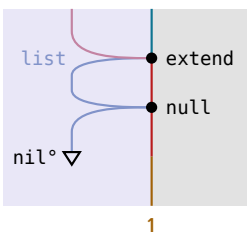
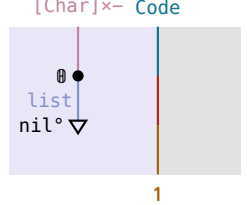
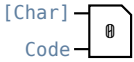
(9.4a)

	name	type	definition	meaning
1	String	Type	$\text{String} \triangleq [\text{Char}]$	a string is a snoc-list of characters
2	LA	Type	$\text{LA} \triangleq [\text{A}]$	a non-empty snoc-list: one value, then any number more appended
3	Code	Type	$\text{Code} \triangleq \text{sym}(\text{Char}) \mid \text{ptr}(\text{String}, \text{String})$	a code is one character, or a pointer back to a repeated piece
4	extend	$\text{String} \times \text{Code} \rightarrow [\text{Char}]$	$\text{extend} \triangleq \text{extend} , \text{extend}((\text{xs}, \text{sym}(\text{a})), \text{x}) = \text{x} = \text{snoc}(\text{xs}, \text{a}) , \text{extend}((\text{xs}, \text{ptr}(\text{ys}, \text{zs})), \text{x}) = \text{x} = \text{cat}(\text{xs}, \text{zs}) \wedge \text{zs} \neq [] \wedge (\text{init}^+)(\text{cat}(\text{ys}, \text{zs}), \text{cat}(\text{xs}, \text{zs}))$	a symbol appends its character; a pointer appends zs when $\text{ys} \# \text{zs}$ is a proper prefix of $\text{xs} \# \text{zs}$
5	decode	$[\text{Code}] \rightarrow [\text{Char}]$	$\text{decode} \triangleq ([\text{nil}, \text{extend}])$	rebuild the string from its codes, left to right; $\text{H} = \text{decode}^\circ$
6	bytes	$\text{Code} \rightarrow \mathbb{N}$	$\text{bytes}(\text{sym}(\text{a})) = \text{c} , \text{bytes}(\text{ptr}(\text{a}, \text{a}')) = \text{p}$	a symbol costs c , a pointer costs p
7	size	$[\text{Code}] \rightarrow \mathbb{N}$	$([\text{nil}, \text{distr} [1 \times \text{c}, 1 \times \text{p}] \text{ plus}]) = \text{size} , \text{size}(\text{wrap}(\text{a})) = 0 , \text{size}(\text{snoc}(\text{cs}, \text{e})) = \text{size}(\text{cs}) + \text{bytes}(\text{e})$	the total cost of a code sequence
8	R	$[\text{Code}] \rightarrow [\text{Code}]$	$\text{R} = \text{size} \leq \text{size}^\circ , \text{R}(\text{xs}, \text{ys}) \iff \text{size}(\text{xs}) \leq \text{size}(\text{ys})$	u costs no more than v
9	prefix	$[\text{Char}] \rightarrow [\text{Char}]$	$\text{xs prefix ys} \iff (\text{prefix}^\circ)(\text{ys}, \text{xs})$	ys is a prefix of x
10	$(\text{T} + \text{T})$	$\text{Code} \rightarrow \text{Code}$	$(\text{T} + \text{T}) = [\text{sym}, \text{ptr}]^\circ (\text{T} + \text{T}) [\text{sym}, \text{ptr}] , \text{sym}(\text{a}) (\text{T} + \text{T}) \text{sym}(\text{a}') \iff \text{true} , \text{ptr}(\text{a}, \text{a}') (\text{T} + \text{T}) \text{ptr}(\text{a}'', \text{a}''') \iff \text{true} , (\forall \text{a}. \forall \text{a}'. \text{e} = \text{sym}(\text{a}) \implies \text{f} = \text{sym}(\text{a}') \implies \text{false}) \wedge (\forall \text{a}. \forall \text{a}'. \forall \text{a}'' . \forall \text{a}''' . \text{e} = \text{ptr}(\text{a}, \text{a}') \implies \text{f} = \text{ptr}(\text{a}'', \text{a}''') \implies \text{false}) \implies (\text{e} (\text{T} + \text{T}) \text{f} \iff \text{false})$	any symbol to any symbol, any pointer to any pointer
11	Q	$\text{F}[\text{Char}] \rightarrow \text{F}[\text{Char}]$	$\text{Q} = \text{F}((\text{T} + \text{T}), \text{prefix}^\circ) , \text{l}(\text{val}) \text{Q} \text{l}(\text{val}') \iff \text{true} , \text{r}(\text{p}) \text{Q} \text{r}(\text{q}) \iff (\text{prefix}^\circ)(\pi_1(\text{p}), \pi_1(\text{q})) \wedge (\text{T} + \text{T})(\pi_2(\text{p}), \pi_2(\text{q})) , (\forall \text{val}. \forall \text{val}' . \text{u} = \text{l}(\text{val}) \implies \text{v} = \text{l}(\text{val}') \implies \text{false}) \wedge (\forall \text{p}. \forall \text{q}. \text{u} = \text{r}(\text{p}) \implies \text{v} = \text{r}(\text{q}) \implies \text{false}) \implies (\text{u} \text{Q} \text{v} \iff \text{false})$	compare two decompositions by the prefix order on the strings and U on the codes
12	reduce	$[\text{Char}] \rightarrow [\text{String} \times \text{Code}]$	$\text{reduce} \triangleq \text{reduce} , \text{q} \in \text{reduce}(\text{ws}) \iff \text{extend}(\text{q}, \text{ws})$	the last code of a string: one symbol, or a pointer to its longest repeated tail

$(\mu X : \frac{[nil, extend]^{\circ}}{\exists} \text{thin}(Q) P([nil, (X \times 1) \text{snoc}]) \text{est}(R)) \sqsubseteq \frac{\text{decode}^{\circ}}{\exists} \text{est}(R) , \quad (9.4b)$ $\text{encode} = (\text{null} \rightarrow nil, \text{reduce list}((\text{encode} \times 1) \text{snoc}) \text{minlist}(R))$ a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded	
circuit	Hinze–Marsden
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E(\text{decode}^{\circ})} P[Code] \xrightarrow{\text{est}(R)} [Code]$ the specification — $\text{decode} \triangleq ([nil, extend])$	
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E([nil, extend]^{\circ})} \text{thin}(Q) \xrightarrow{PF[Char]} P([nil, (X \times 1) \text{snoc}]) \xrightarrow{P[Code]} \text{est}(R) \xrightarrow{[Code]}$ at $Q \triangleq F(T+T, \text{prefix}^{\circ}) = 1 + (\text{prefix}^{\circ} \times (T+T))$, the two T on symbols and on pointers. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(T+T)[c, p] = [c, p]$. $\sqsubseteq$ Proposition 9.4 splits the thinning condition in two — $F(T+T, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^{\circ} \text{decode}$, for which $\text{decode init} \sqsubseteq R^{\circ} \text{decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins	
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E(\text{extend}^{\circ})} \text{thin}(\text{prefix}^{\circ} \times (T+T)) \xrightarrow{P([Char] \times \text{Code})} P((X \times 1) \text{snoc}) \xrightarrow{P[Code]} \text{est}(R) \xrightarrow{[Code]}$ $\sqsubseteq$ Proposition 9.1: nil and $extend$ have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^{\circ}}{\exists} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr}(ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer	
$[Char] \xrightarrow{\text{reduce}} [[Char] \times \text{Code}] \xrightarrow{\text{list}((\text{encode} \times 1) \text{snoc})} [[Code]] \xrightarrow{\text{minlist}(R)} [Code]$ $\sqsubseteq$ reduce implements $\frac{\text{extend}^{\circ}}{\exists} \text{thin}(\text{prefix}^{\circ} \times (T+T))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt. Again exponential; the book gives no tabulation for it	

§9.4.1. Derivation

$Q \vdash F(\text{decode}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ) [\text{nil}, \text{snoc}] R$ (9.4.1a)	
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}, r(p) \ Q \ r(q) \iff (\text{prefix}^\circ)(\pi_1(p), \pi_1(q)) \wedge (T+T)(\pi_2(p), \pi_2(q)), (\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})$ $\implies (u \ Q \ v \iff \text{false})$	
$=$ the bifunctor F preserves composition: $F(U, \text{prefix}^\circ) F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last character drops the last sym, or shortens or drops the last pointer. Proved at prefix directly by induction, not through init	
$=$ F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ) F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: snoc adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$	

extend nil[°] = 0 (9.4.1b) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
	
 nil returns only [], so $\text{nil}^\circ = \text{null } \text{nil}^\circ$	
 $\text{extend } \text{null} = 0$: a symbol ends the string with a character, a pointer with its non-empty ZS	
 0 composed with anything is 0	